

DAY 2 — Routing, Data-Centre Networking, VXLAN/EVPN, WAN and SD-WAN

The central idea for today is:

Modern networks separate physical connectivity from logical connectivity, and they separate learning decisions from packet forwarding.

Once that idea is clear, technologies such as **BGP, VRF, MPLS, VXLAN, EVPN, leaf-spine and SD-WAN** stop looking like unrelated acronyms.

A useful evolution is:

Simple LAN
↓
Routers + static routes
↓
Dynamic routing: OSPF / BGP
↓
Large enterprise / WAN: VRF + MPLS
↓
Modern DC: Leaf-Spine + ECMP
↓
IP Underlay
↓
VXLAN Overlay
↓
BGP EVPN Control Plane
↓
SD-WAN / Hybrid Cloud connectivity

1. Routing fundamentals

What is routing?

Routing answers:

Where should an IP packet go next to eventually reach its destination?

Suppose:

Laptop
10.1.1.20
|
Router-A
|
Router-B
|
Server
10.5.10.50

The laptop knows that 10.5.10.50 is outside its local subnet, so it sends the packet to its **default gateway**.

Router-A examines the destination IP address and asks:

Do I know how to reach 10.5.10.50?

It consults its **routing table**.

Routing table

A routing table is essentially a database containing knowledge such as:

Destination	Next Hop	Interface
10.1.0.0/16	directly	eth0
10.5.10.0/24	172.16.1.2	eth1
10.5.0.0/16	172.16.2.2	eth2
0.0.0.0/0	192.0.2.1	eth3

It does not normally contain one entry for every individual host.

It normally contains **network prefixes**.

Route

A route is a piece of information saying:

For this destination prefix,
send traffic toward this next hop/interface.

Example:

10.5.10.0/24 → next hop 172.16.1.2

Prefix

A prefix represents a range of IP addresses.

For example:

10.5.10.0/24

means roughly:

10.5.10.0
through
10.5.10.255

The /24 tells us how many bits describe the network portion.

Next hop

The next hop is usually the next router toward the destination.

A router doesn't necessarily know the entire end-to-end physical route.

It only needs to know:

Where should I send this packet next?

This is an extremely important networking idea.

What if multiple routes match?

Suppose a router has:

10.0.0.0/8
10.5.0.0/16
10.5.10.0/24
0.0.0.0/0

Destination:

10.5.10.25

Technically all four routes match.

The router chooses:

10.5.10.0/24

because routers normally use:

Longest Prefix Match

The most-specific matching prefix wins.

/24 beats /16
/16 beats /8
/8 beats /0

Think:

| Use the most precise direction available.

Metric

Sometimes several routes exist to the **same prefix**.

Example:

10.5.10.0/24 via Router-A
10.5.10.0/24 via Router-B

A routing protocol may assign costs to paths.

These are broadly called **metrics**.

For example, OSPF uses a cost concept.

Lower-cost path:

Router → A → Destination cost 20
Router → B → C → Destination cost 50

Choose A.

Administrative preference

There can also be routes to the same destination learned from **different sources**.

For example:

Static route
OSPF route
BGP route

The router may have a preference for one route source over another.

Different vendors use terms such as:

- administrative distance
- route preference
- preference value

The exact numbers are implementation/vendor dependent.

Conceptually:

Which SOURCE of routing information do I trust more?

Do not confuse:

Administrative preference

↓

chooses between routing sources

Metric

↓

usually chooses paths within a routing protocol

Default route

A default route means:

If you don't know anything more specific,
send the packet here.

IPv4:

0.0.0.0/0

IPv6:

::/0

A home router, for example, normally has a default route toward the ISP.

Static route

An administrator manually defines:

10.50.0.0/16 → Router-B

Advantages

Simple and predictable.

Problem

Imagine maintaining:

20 routers
200 networks
multiple redundant paths
frequent failures

Manual routing becomes difficult.

Dynamic route

A routing protocol automatically exchanges reachability information.

Routers can learn:

10.50.0.0/16 reachable through Router-B
10.60.0.0/16 reachable through Router-C

If connectivity changes, routing protocols update their knowledge.

Route convergence

Imagine:

```
A ---- B ---- D
 \          /
  ---- C ----
```

Traffic initially travels:

A → B → D

Then link:

B → D

fails.

The network must:

1. detect the failure,
2. update routing information,
3. calculate another path,
4. install new forwarding information.

Eventually:

A → C → D

That process is **convergence**.

Fast convergence matters because packets may be dropped during the transition.

2. Why dynamic routing protocols exist

Consider a large organisation:

Bangalore Office
Mumbai Office
London Office
AWS
Azure
Two Data Centres
100+ routers

With static routing, every topology change could require human modifications.

Problem

Networks change continuously.

Links fail.

New networks are added.

Cloud routes appear.

Routers are replaced.

Earlier solution

Static routes.

Limitation

Poor scalability and slow operational response.

Modern solution

Dynamic routing protocols.

Two extremely important ones are:

OSPF
BGP

But they solve somewhat different problems.

OSPF vs BGP

Area	OSPF	BGP
Primary role	Routing inside an organisation/ network domain	Routing between routing domains and policy- heavy environments
Type	Link-state IGP	Path-vector inter-domain protocol
Main concern	Finding good internal paths	Choosing routes based on reachability + policy
Topology knowledge	Strong topology awareness	Doesn't build an OSPF-style complete topology map
Common use	Enterprise/core/internal networks	Internet, cloud edges, large DC fabrics, EVPN
Path decision	Primarily cost	Multiple path attributes/policies
Scale model	Internal routing	Huge routing/policy scale

A simplified mental model:

OSPF:

"What is the shortest/best internal path?"

BGP:

"Which advertised path do policy and attributes say I should use?"

Basic dynamic-routing vocabulary

Neighbour / peer

Routers establish relationships with other routers.

Router-A ← routing relationship → Router-B

In BGP the term **peer** is very common.

Route advertisement

Router-A tells Router-B:

"I can provide reachability to 10.20.0.0/16."

Route learning

Router-B receives the advertisement and learns:

10.20.0.0/16 → Router-A

Route withdrawal

If Router-A can no longer provide that reachability, it can withdraw the route.

Conceptually:

Earlier:

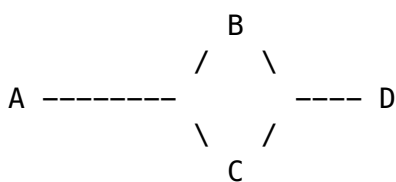
10.20.0.0/16 → available

Later:

10.20.0.0/16 → withdrawn

Path selection

If several paths exist:



the routing protocol determines which path/path(s) should be used.

3. OSPF

OSPF stands for:

Open Shortest Path First

It is primarily an **Interior Gateway Protocol**, meaning that it is generally used for routing inside an administrative network.

Problem

Suppose your enterprise contains:



R1 needs to determine good paths to all internal networks.

Static routes would require extensive manual configuration.

Link-state concept

Instead of merely telling neighbours:

"I know network X."

OSPF routers exchange information about their connectivity.

Conceptually:

R1:

I connect to R2 and R3.

R2:

I connect to R1 and R4.

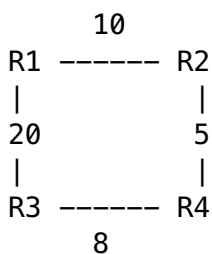
R3:

I connect to R1 and R5.

Routers build an internal representation of network topology.

Topology awareness

OSPF therefore has a picture resembling:



The numbers represent costs.

Each router can calculate paths.

Shortest-path calculation

OSPF uses a shortest-path algorithm conceptually based on Dijkstra's algorithm.

You do not need packet-level details to understand its purpose.

Think:

Topology information
↓
Calculate shortest-path tree
↓
Select best next hops
↓
Install routes

Where OSPF is useful

Typical use:

Enterprise internal network
Campus routing
WAN/internal backbone
Infrastructure networks
Some data-centre underlays

At very large scale, OSPF designs may use **areas** to reduce topology/state scope.

The important distinction is:

| OSPF is mainly about efficiently finding paths through an internal topology.

4. BGP

BGP stands for:

Border Gateway Protocol

It is one of the most important protocols in modern networking.

The Internet relies heavily on BGP for inter-domain routing. BGP is defined fundamentally as an inter-Autonomous-System routing protocol.

Autonomous System

An Autonomous System, or **AS**, is roughly:

| A collection of networks operated under a common routing administration/policy.

For example:

ISP-A	AS 65001
ISP-B	AS 65002
Enterprise	AS 65010

Public Internet AS numbers are allocated according to Internet-numbering processes, while private AS ranges also exist for internal use.

eBGP

External BGP is used between different Autonomous Systems.

AS 65001 AS 65002
Router-A ← eBGP → Router-B

iBGP

Internal BGP exchanges BGP information between routers belonging to the same AS.

AS 65001
Router-A ← iBGP → Router-B

BGP advertisements

Imagine:

Company-A
203.0.113.0/24
|
ISP-A

Company-A/its provider can advertise reachability:

203.0.113.0/24

Other BGP routers propagate routing information according to their policies.

BGP is policy-driven

This is what makes BGP fundamentally different from the simplistic idea:

| Pick the shortest physical route.

BGP considers attributes and routing policy.

Conceptually attributes can tell routers things such as:

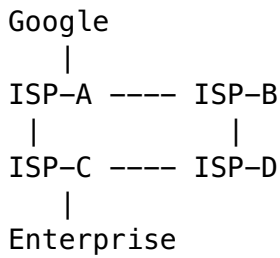
Which Autonomous Systems has this route passed through?
Which path should I prefer internally?
Which neighbouring path should be preferred?
How should this route be treated?

For example, AS_PATH records ASes through which advertised routing information has passed. BGP supports several classes of path attributes. ([RFC Editor](#))

You do not need to memorize the selection algorithm yet.

Why BGP is used on the Internet

Imagine:



There is no single organisation controlling all these networks.

Routing therefore requires:

- distributed route exchange,
- enormous scale,
- routing policies,
- independent administrative domains.

BGP solves this problem.

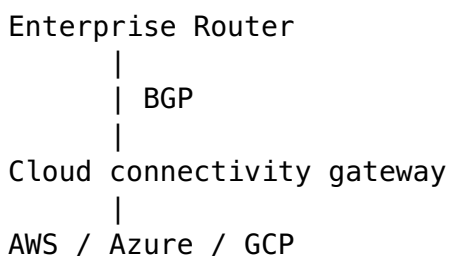
BGP in large enterprises

An enterprise may use BGP for:

Internet connections
Cloud connections
Multiple data centres
Multi-homing
Large internal routing domains
MPLS/VPN environments

BGP in cloud networking

For example:



Dedicated connections and dynamic VPN scenarios commonly use BGP to exchange prefixes.

The important architectural idea is:

Enterprise tells cloud:
"These prefixes are reachable through me."

Cloud tells enterprise:
"These prefixes are reachable through me."

BGP in modern data centres

Modern leaf-spine networks frequently use BGP because:

- it scales well,
- policy is powerful,
- it supports multipath designs,
- operators can use it for the underlay,
- BGP extensions can carry EVPN information.

This produces an important distinction:

BGP IPv4/IPv6 routing
≠
BGP EVPN

Same protocol family,
different information being distributed.

5. ECMP and BFD

ECMP

ECMP = **E**qual-**C**ost **M**ulti-**P**ath

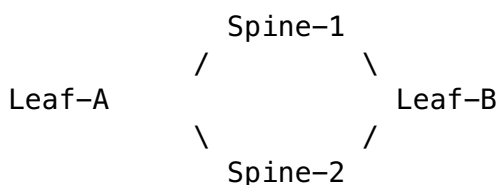
Old idea:

Use one best path.

Modern data-centre requirement:

Why leave equivalent paths unused?

Suppose:



Both paths can have the same routing cost.

Instead of:

Leaf-A → Spine-1 → Leaf-B

only, ECMP can use:

Leaf-A → Spine-1 → Leaf-B

AND

Leaf-A → Spine-2 → Leaf-B

Why ECMP matters

It provides:

More bandwidth

Multiple links can carry traffic.

Better infrastructure utilisation

Redundant links don't have to sit completely idle.

Resilience

If a path disappears, another remains available.

Traffic is typically distributed according to a hash derived from packet-flow fields, helping packets belonging to one flow stay on a consistent path.

BFD

BFD = **Bidirectional Forwarding Detection**.

Problem:

Routing protocol timers may take longer than desired to decide that a forwarding path failed.

BFD provides a lightweight mechanism designed for rapid detection of forwarding-path failures and can operate independently of the routing protocol itself.

Conceptually:

Router A ===== Router B
BFD

"Are you alive?"
"Yes."

"Are you alive?"
(no response)

↓
Path declared unavailable
↓
Routing reacts

BFD does not calculate routes.

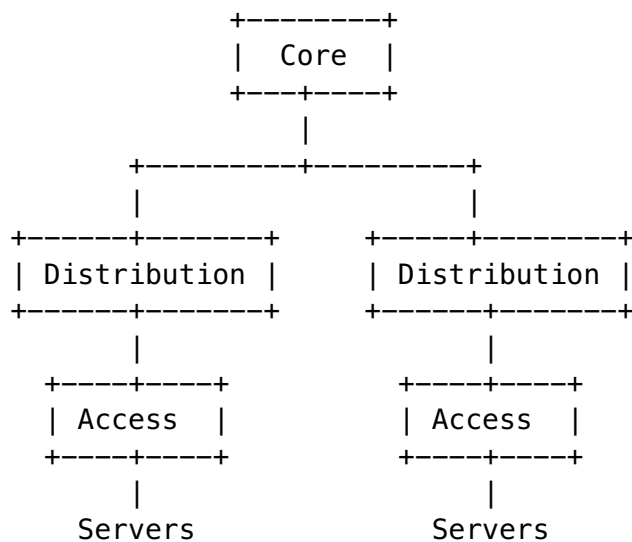
It helps detect failures quickly.

Think:

BFD → detect failure
OSPF/BGP → react/recalculate routes

6. Traditional data-centre architecture

Historically many enterprise data centres used three tiers:



Access layer

Servers connect here.

Distribution / aggregation layer

Aggregates access switches and commonly handles routing/policies.

Core

Provides high-speed connectivity between major network sections.

Why this architecture worked

Traditional enterprise applications often generated substantial:

north-south traffic

User
↓
Data centre
↓
Application

But modern distributed applications changed traffic patterns.

East-west traffic

Consider a microservice application:

API
↓
Service A
↓
Service B
↓
Cache
↓
Database

A request may move repeatedly between servers inside the data centre.

That is **east-west traffic**.

Limitation of traditional hierarchy

Traffic could take awkward paths through aggregation/core layers.

Scaling could also mean:

bigger switches
more complex spanning trees
larger failure domains

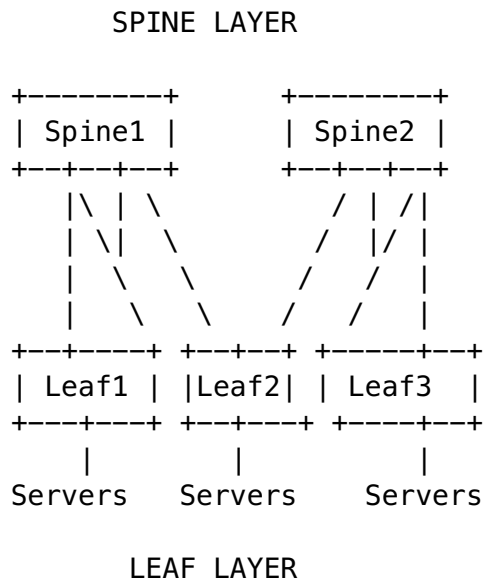
Modern data centres wanted:

many parallel paths
predictable latency
horizontal scale
simple L3 fabric

That led to leaf-spine.

7. Leaf-spine architecture

Consider:



Every leaf normally connects to every spine.

Leaves normally do not need direct leaf-to-leaf physical links for ordinary fabric connectivity.

Spines normally don't need direct spine-to-spine forwarding links inside the basic fabric.

Leaf switches

Servers, hypervisors, appliances and other endpoints connect to leaves.

In many designs:

Leaf = Top-of-Rack switch

although implementations vary.

Spine switches

Spines provide high-speed connectivity between leaves.

Predictable path

Server on Leaf1 to server on Leaf3:

```
Server
  ↓
Leaf1
  ↓
Spine
  ↓
Leaf3
  ↓
Server
```

Usually:

Leaf → Spine → Leaf

Predictable hop count.

ECMP

With four spines:

```
Leaf1 → Spine1 → Leaf2
Leaf1 → Spine2 → Leaf2
Leaf1 → Spine3 → Leaf2
Leaf1 → Spine4 → Leaf2
```

All can potentially participate in forwarding.

Horizontal scaling

Need more server capacity?

Add another leaf:

```
      S1 S2 S3 S4
     /|\ | /| /|
    ...
      |
     L5
      |
    new servers
```

Need additional fabric bandwidth?

Subject to platform/design limits, add spine capacity or higher-speed links.

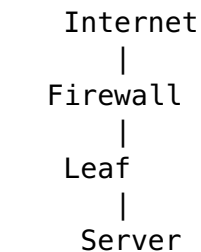
The architectural preference moves away from:

One enormous hierarchical switch

toward:

many standardized components
+
parallel paths

North-south vs east-west



NORTH-SOUTH

versus:

Server A → Leaf → Spine → Leaf → Server B

EAST-WEST

Modern distributed systems create huge amounts of east-west communication, which is one reason leaf-spine architectures became common.

8. Control plane vs data plane vs management plane

This distinction is fundamental for platform/network automation engineers.

Control plane

The control plane decides:

| What forwarding information should exist?

Examples:

OSPF
BGP
EVPN

Conceptually:

Learn topology/routes
↓
Select paths
↓
Build forwarding information

Data plane

The data plane performs:

| Forward this actual packet.

Example:

```
Packet arrives
destination 10.20.1.10
      ↓
look up forwarding table
      ↓
send out interface 7
```

The data plane must operate extremely quickly.

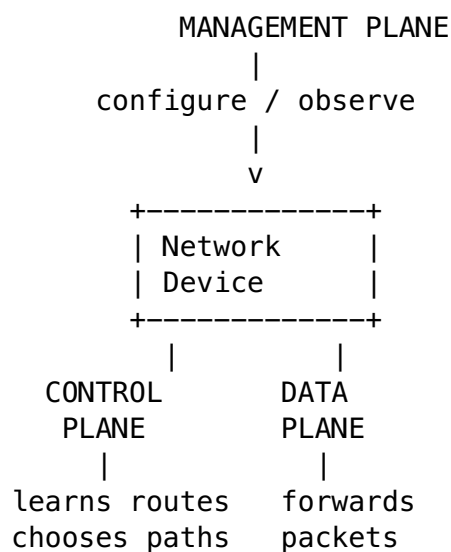
Management plane

The management plane is how humans and automation systems operate network devices.

Examples include:

```
APIs
SSH
NETCONF
RESTCONF
gNMI
telemetry
configuration systems
monitoring
```

The three together



Connection to network automation

A network automation system normally interacts primarily with:

Management plane

For example:

Python automation

|
| API
v

Network device

|
configure BGP
configure VRF
retrieve routes
collect telemetry

That configuration influences:

Control plane

which ultimately changes:

Data plane

So:

Automation

↓

Management plane

↓

Control plane

↓

Forwarding state

↓

Data plane

This chain is worth remembering.

9. VRF

VRF = **Virtual Routing and Forwarding**.

Think:

| Multiple independent routing tables inside one router.

Without VRF:

Router

Routing Table

10.1.0.0/16

10.2.0.0/16

10.3.0.0/16

With VRFs:

Router

|
+---- VRF Finance

| 10.1.0.0/16

|
+---- VRF Engineering

| 10.1.0.0/16

|
+---- VRF Customer-A

 10.1.0.0/16

Notice that even overlapping addresses can exist because each VRF has an independent routing context.

Why VRF exists

Problem

One physical network device may support multiple isolated networks.

Earlier approach

Use separate physical routers.

Finance → Router1

HR → Router2

Guest → Router3

Limitation

Expensive and operationally inefficient.

Solution

Virtualise the routing tables.

Physical Router

```
+-----+
| VRF Finance |
| VRF Engineering |
| VRF Guest   |
+-----+
```

Enterprise use

Examples:

Corporate
Guest
Production
Development
Management

can maintain separate routing domains.

Service-provider use

A provider can carry traffic for many customers:

Customer A → VRF-A
Customer B → VRF-B
Customer C → VRF-C

while keeping routes logically isolated.

Data-centre use

VRFs commonly provide tenant or application segmentation:

Tenant A
|
VRF-A
|
VLAN/VXLAN networks

Tenant B
|
VRF-B

This concept becomes important again when we reach VXLAN/EVPN.

10. MPLS fundamentals

MPLS = **M**ultiprotocol **L**abel **S**witching.

Historically, service providers needed scalable ways to transport customer traffic and build managed VPN services.

Basic IP forwarding

Normal routing resembles:

Packet
destination 10.5.5.5
↓
router examines destination
↓
routing/forwarding lookup
↓
next hop

MPLS idea

Within an MPLS domain, forwarding can use a **label** associated with the traffic.

Conceptually:

IP packet
↓
attach MPLS label
↓
Provider MPLS network
↓
forward based on labels
↓
exit provider network

You do not need label-stack mechanics today.

MPLS VPN

Service providers could offer:

Company Branch A
|
|
Provider
MPLS Core
|
|
Company Branch B

Customer routes can be kept logically isolated.

A conceptual architecture:

Provider MPLS Network

Branch-A -- PE ===== PE -- Branch-B

 | |

 VRF VRF

 Customer-X Customer-X

PE here means provider-edge router.

Important clarification

MPLS VPN does **not automatically mean encryption**.

It primarily provides traffic separation and managed forwarding inside the provider network.

If encryption is required, it must be provided separately or by the particular service design.

Why MPLS became popular for enterprise WAN

It offered:

Private managed WAN connectivity

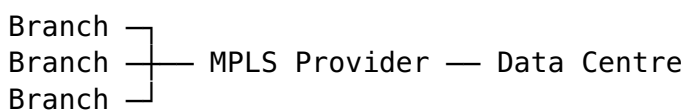
Predictable provider services

QoS possibilities

Many-to-many connectivity

Routing integration

For years, enterprise architectures commonly looked like:



But MPLS could be relatively expensive and dependent on provider provisioning.

Internet connectivity became much cheaper and widely available.

That helps explain SD-WAN later.

11. Underlay and overlay networks

This is one of today's most important topics.

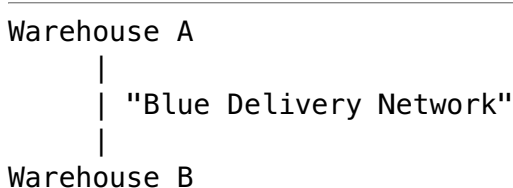
Analogy

Imagine cities connected through highways.

Bangalore ===== Hyderabad ===== Delhi

That highway system is the **underlay**.

Now imagine a logistics company defines a virtual delivery service:



The delivery service uses the highways but presents its own logical network.

That is the **overlay**.

Underlay

The underlay provides basic physical/IP reachability.

In a leaf-spine data centre:

```
Server
 |
Leaf1
 / \
S1  S2
 \ /
Leaf2
```

The underlay answers:

Can Leaf1 reach Leaf2's IP address?

Typically the underlay is an IP routed network.

Overlay

The overlay creates logical connectivity over that IP fabric.

Example:

```
VM-A ----- VM-B
      logical tenant network
```

although physically:

```
VM-A
 |
Leaf1
 |
Spine1
 |
Leaf2
 |
VM-B
```

Why separation helps

The underlay does not need detailed knowledge about every application's logical segmentation.

It mainly needs to provide:

Reliable IP reachability between tunnel endpoints.

The overlay can independently express:

Tenant A
Tenant B
Application networks
Security zones
Logical Layer-2 networks
Logical Layer-3 networks

Technical example

Underlay:

Leaf1 VTEP = 10.255.1.1
Leaf2 VTEP = 10.255.1.2

10.255.1.1 can reach 10.255.1.2

Overlay:

Tenant-A VNI 10010

VM-A 192.168.1.10
VM-B 192.168.1.20

VM traffic is encapsulated between:

10.255.1.1 → 10.255.1.2

The physical fabric routes the outer IP packet without needing to treat the tenant addresses as the fabric topology.

Operational advantage

You can potentially change:

physical links
routing topology
spines
underlay addresses

without redesigning every tenant's logical network.

Likewise, logical tenant networks can be created without creating a matching physical topology.

That separation is fundamental to modern cloud networking.

12. Why VLANs became limiting

VLANs solve a useful problem:

| Divide a Layer-2 network into multiple isolated broadcast domains.

Example:

VLAN 10 = Finance
VLAN 20 = Engineering
VLAN 30 = Management

VLAN problem 1: identifier scale

Classic VLAN identification is based on a 12-bit VLAN ID field, giving roughly 4,000 usable network segments.

That may sound large for an office.

It can be restrictive for:

large cloud platforms
multi-tenant environments
huge virtualized data centres

VXLAN explicitly addressed VLAN-range and traditional L2 scaling limitations. ([RFC Editor](#))

VLAN problem 2: Layer-2 domain scale

Large Layer-2 environments require handling:

broadcast
unknown unicast
multicast
MAC learning
loop prevention

Extending giant L2 domains throughout a data centre increases operational complexity.

VLAN problem 3: workload mobility

Suppose:

VM-A
VLAN 100
Rack 1

moves to:

Rack 200

Maintaining identical Layer-2 connectivity everywhere using traditional mechanisms can become cumbersome.

VLAN problem 4: multi-tenancy

Imagine a public cloud with:

Customer 1 → 500 networks

Customer 2 → 300 networks

Customer 3 → 700 networks

...

10,000 customers

A VLAN-only segmentation model becomes restrictive.

Enter VXLAN

The design requirement becomes:

Keep scalable IP routing in the physical network while providing large numbers of logical network segments above it.

13. VXLAN

VXLAN = **V**irtual **eX**tensible LAN.

Its fundamental model is:

Layer-2 network

↓

encapsulated

↓

carried through Layer-3 IP network

The VXLAN specification describes an overlay for virtual Layer-2 networks over a Layer-3 network. ([RFC Editor](#))

VNI

VNI = **VXLAN Network Identifier**.

Think of it somewhat like an expanded logical segmentation identifier.

Example:

VNI 10001 = Tenant A / Web

VNI 10002 = Tenant A / Database

VNI 50001 = Tenant B / Web

A VXLAN VNI is 24 bits, allowing about **16 million VXLAN segments**. ([RFC Editor](#))

Contrast:

VLAN ID
12 bits
≈ 4K segments

VNI
24 bits
≈ 16 million segments

This is a major scale increase.

VTEP

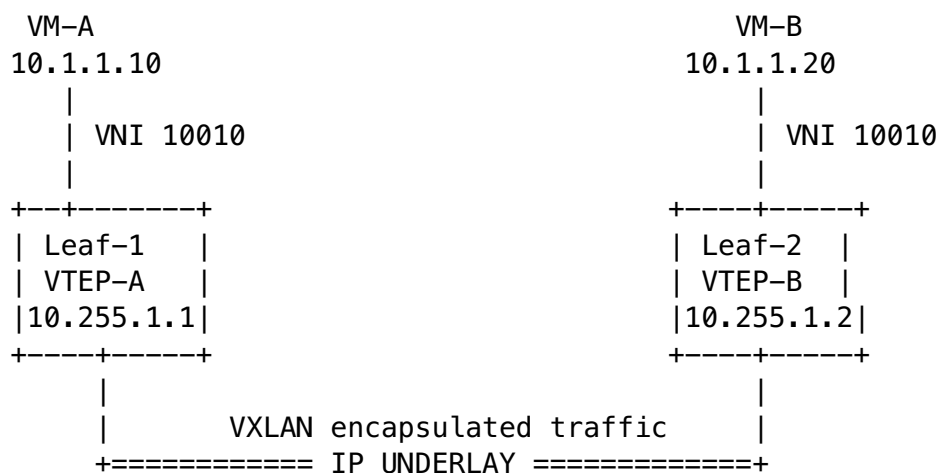
VTEP = **VXLAN Tunnel Endpoint**.

The VTEP performs:

encapsulation
decapsulation

Modern leaf switches commonly act as VTEPs.

Basic VTEP architecture



What happens?

VM-A sends:

Ethernet frame
src MAC = VM-A
dst MAC = VM-B

Leaf1/VTEP-A determines that VM-B is reachable through VTEP-B.

It encapsulates the original frame.

Conceptually:

Outer IP:
source = VTEP-A
destination = VTEP-B

VXLAN:
VNI = 10010

Inner:
VM-A Ethernet frame → VM-B

The spine routers only need to route:

VTEP-A IP → VTEP-B IP

They do not need ordinary underlay routes for every VM.

At VTEP-B

VXLAN packet
↓
remove outer headers
↓
recover original Ethernet frame
↓
deliver to VM-B

Why VXLAN scales

Because two problems are separated:

Physical scalability
↓
IP routed fabric

Logical tenant scalability
↓
VNI overlay

VXLAN does NOT itself solve everything

VXLAN tells us:

| **How should we transport encapsulated data?**

But there is another question:

How does VTEP-A know
that VM-B is behind VTEP-B?

That brings us to EVPN.

14. EVPN

EVPN = **E**thernet **V**PN.

EVPN is fundamentally a **control-plane approach for distributing Ethernet/IP reachability information**.

Problem before EVPN

Suppose Leaf1 receives a frame destined for:

MAC BB:BB:BB:BB:BB:BB

How does Leaf1 know where that MAC lives?

Traditional Ethernet uses mechanisms involving:

MAC learning
broadcast
unknown-unicast flooding
ARP

Across a huge overlay, excessive flooding is undesirable.

EVPN approach

Rather than relying purely on:

"Send traffic everywhere and discover where the endpoint lives."

the network can distribute endpoint reachability through a control plane.

Conceptually:

Leaf2 learns locally:

VM-B
MAC = BB:BB...
IP = 10.1.1.20

Leaf2
|
| BGP EVPN advertisement
v
network
|
v

Leaf1 learns:

BB:BB... / 10.1.1.20
reachable through Leaf2

RFC 7432 defines EVPN control-plane learning where remote MAC information can be advertised using MP-BGP MAC/IP Advertisement routes. ([RFC Editor](#))

BGP EVPN

BGP was extended to carry EVPN information.

This does not mean BGP has suddenly become Ethernet forwarding itself.

Instead:

BGP

↓

transport/distribute EVPN reachability

Information can include concepts such as:

MAC address

IP address

which network/tenant

which VTEP can reach it

EVPN route types

You don't need to memorize them today.

At high level, EVPN can distribute information concerning:

endpoint MAC/IP reachability

multi-homing

BUM/flooding participation

IP prefix reachability

For orientation only, you'll commonly encounter names such as:

Type 2 → MAC/IP advertisement

Type 3 → inclusive multicast information

Type 5 → IP prefix

The important thing today is their purpose rather than route-number memorisation.

Why this is better

Instead of relying exclusively on data-plane discovery:

Flood → discover → learn

you increasingly use:

Learn locally
↓
advertise through control plane
↓
remote VTEPs learn proactively

Flooding does not disappear completely because some broadcast/unknown/multicast traffic still exists, but reliance on blind flooding can be substantially reduced.

15. VXLAN + EVPN

This distinction is extremely important:

```
+-----+
| EVPN          |
| CONTROL PLANE |
|               |
| "Where is endpoint X?" |
+-----+
```

↓

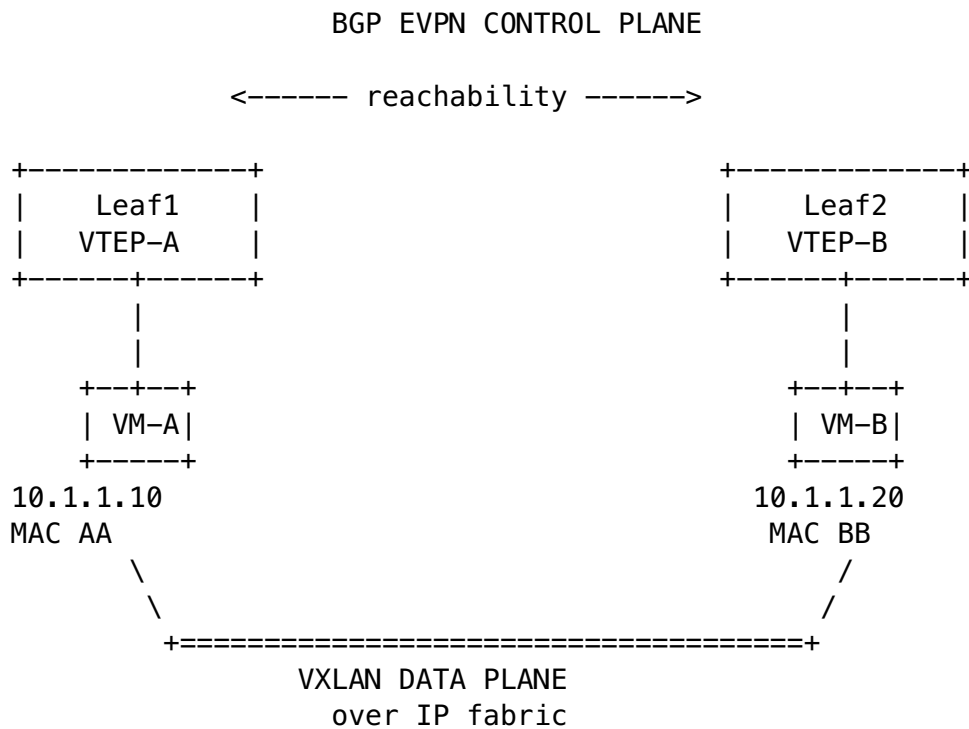
```
+-----+
| VXLAN         |
| DATA-PLANE ENCAPSULATION |
|               |
| "How do I carry its Ethernet traffic?" |
+-----+
```

A compact memory aid:

| EVPN tells you WHERE. VXLAN carries the DATA there.

RFC 8365 specifically describes EVPN as a network-virtualization-overlay control-plane solution with encapsulations including VXLAN over an IP underlay.

Architecture



Full packet walkthrough

Assume:

VM-A
IP 10.1.1.10
MAC AA
Leaf1 / VTEP 10.255.1.1

VM-B
IP 10.1.1.20
MAC BB
Leaf2 / VTEP 10.255.1.2

VNI 10010

Step 1 — VM-B connects to Leaf2

Leaf2 learns locally:

MAC BB
IP 10.1.1.20
VNI 10010

Step 2 — EVPN advertises reachability

Conceptually Leaf2 tells Leaf1:

Endpoint:

MAC BB
IP 10.1.1.20
VNI 10010

is reachable through:

VTEP 10.255.1.2

Step 3 — VM-A sends traffic

VM-A generates an Ethernet frame toward VM-B.

Step 4 — Leaf1 consults EVPN-derived information

Leaf1 determines:

VM-B → VTEP-B

Step 5 — VXLAN encapsulation

Leaf1 encapsulates:

Outer source IP:
10.255.1.1

Outer destination IP:
10.255.1.2

VNI:
10010

Inner frame:
AA → BB

Step 6 — IP underlay routes it

The spine network doesn't need to care that:

VM-A = 10.1.1.10
VM-B = 10.1.1.20

It primarily needs:

How do I reach 10.255.1.2?

Step 7 — Leaf2 decapsulates

Leaf2 removes VXLAN encapsulation.

Step 8 — VM-B receives original frame

Done.

Where VRF fits

A modern fabric may have:

VRF-A

|
+--- VNI 10010 Web
|
+--- VNI 10020 DB

VRF-B

|
+--- VNI 20010 Web

So you can think:

VRF

=
Layer-3 routing isolation

VNI

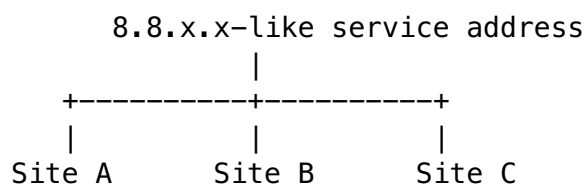
=
overlay network identification

There are designs involving both Layer-2 VNIs and Layer-3 VNIs, but we don't need their implementation details yet.

16. Anycast

Anycast means multiple systems provide the **same logical address**, with routing directing traffic toward an appropriate reachable instance.

Simple example:



Clients don't necessarily need to know:

Server-A IP
Server-B IP
Server-C IP

They use the service address.

Routing determines where traffic goes.

DNS example

Large DNS services can advertise the same address from many locations.

A Bangalore user might reach:

nearby Indian/Asian infrastructure

while a London user reaches:

European infrastructure

using the same advertised service IP.

Data-centre anycast gateway

VXLAN/EVPN fabrics frequently use the idea of an **anycast default gateway**.

Suppose:

VM-A on Leaf1

VM-B on Leaf5

gateway = 10.1.1.1

Multiple leaves can present the same gateway identity locally.

Conceptually:

Gateway 10.1.1.1			
Leaf1	Leaf2	Leaf3	Leaf4
same	same	same	same
logical	logical	logical	logical
gateway	gateway	gateway	gateway

This lets workloads use a local leaf gateway rather than hairpinning traffic to one central router.

17. WAN fundamentals

WAN = **Wide Area Network**.

A WAN connects geographically separated networks.

Example:

Bangalore
|
Mumbai
|
London
|
Data Centre

Typical WAN connectivity

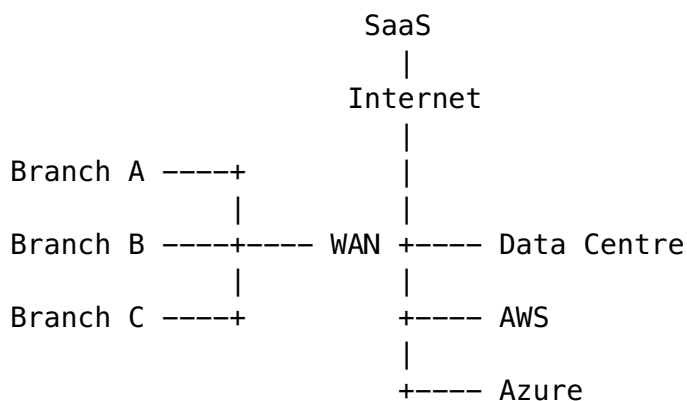
Historically:

Private leased circuits
MPLS provider networks

Increasingly:

Internet broadband
DIA
5G/LTE
Cloud connectivity
VPN
SD-WAN

Enterprise example



A WAN design needs to answer:

How do branches reach each other?
How do branches reach clouds?
How is traffic secured?
What happens when a link fails?
Which path should an application use?

Redundant WAN paths

A branch could have:

MPLS
+
Internet
+
5G backup

instead of depending on one circuit.

This idea becomes fundamental in SD-WAN.

18. VPN concepts

VPN = **Virtual Private Network**.

It creates logical private connectivity across another network.

Site-to-site VPN

Connects networks.

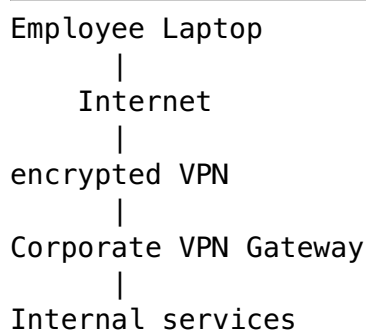


The tunnel is generally encrypted.

Users inside the networks may not be aware that their packets traverse a VPN.

Remote-access VPN

Connects an individual user's device to an organisation.



Why tunnels?

The Internet is an untrusted shared network.

Instead of exposing internal traffic directly:

Private packet
↓
encrypt + encapsulate
↓
Internet
↓
decapsulate + decrypt

Tunnelling appears repeatedly in networking:

IPsec VPN
VXLAN
GRE
SD-WAN

but the purpose is not always identical.

Important:

Tunnel ≠ automatically encryption.

VXLAN normally provides encapsulation,
not security encryption by itself.

IPsec provides encryption/authentication.

19. SD-WAN

SD-WAN = **Software-Defined Wide Area Network**.

To understand SD-WAN, start with traditional WAN pain.

Traditional WAN

Imagine 500 branches.

Branch-1 router
Branch-2 router
Branch-3 router
...
Branch-500 router

Each branch might require:

routing configuration
VPN configuration
QoS
failover rules
security policy
provider circuits

Operating this box-by-box becomes difficult.

Traditional application routing

Suppose Branch-A has:

MPLS
Internet

Historically traffic might be statically designed:

Everything critical → MPLS
Backup → Internet

But applications differ.

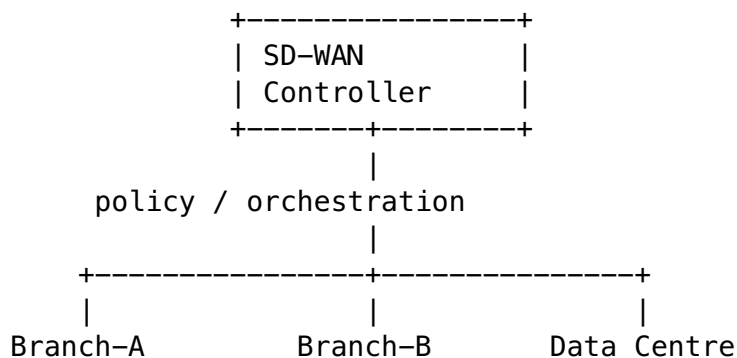
For example:

Teams
Office 365
SAP
VoIP
YouTube
internal finance system

They don't all have the same network requirements.

SD-WAN idea

Create a centrally managed WAN overlay.



The controller does not normally forward every user packet.

Forwarding remains distributed in edge devices.

So "centralised" primarily means:

central policy
central visibility
central orchestration

rather than:

all traffic must pass through one controller

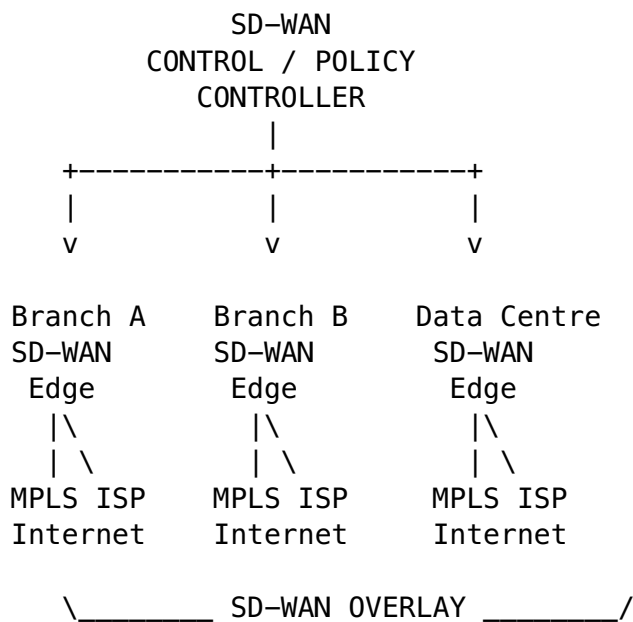
Multiple transports

An SD-WAN edge may have:

MPLS
Internet ISP-1
Internet ISP-2
LTE/5G

The overlay can use these transports underneath it.

SD-WAN architecture



Application awareness

Suppose two paths exist.

MPLS:
latency 25ms
loss 0%

Internet:
latency 40ms
loss 0.2%

Voice might prefer:

MPLS

A SaaS application might go directly:

Internet → SaaS

instead of:

Branch
↓
MPLS
↓
Data centre
↓
Internet
↓
SaaS

This direct breakout can improve efficiency.

Dynamic path selection

Suppose MPLS develops:

packet loss = 8%

while Internet remains healthy.

Policy might move voice traffic to Internet.

Conceptually:

Measure paths
↓
Understand application/policy
↓
Choose suitable path
↓
Continuously reevaluate

Traditional WAN vs SD-WAN

Traditional WAN	SD-WAN
Often device/provider centric	More centrally policy-driven
MPLS often dominant	MPLS + Internet + broadband + cellular
Static/path-centric policies common	Application-aware policies
Box-by-box management can be common	Central orchestration
Internet backup common	Internet can be active transport
Change may require considerable manual/provider work	Policy can often be distributed centrally
Limited unified visibility	Centralised telemetry/visibility common

20. WLAN fundamentals

WLAN = Wireless Local Area Network.

The basic endpoint is an **access point**.

```
Laptop
  )))
Access Point
  |
Wired Network
```

SSID

SSID is the wireless network name users see.

Example:

```
Company-Corporate
Company-Guest
```

But an SSID is not itself the complete security model.

Authentication

Enterprise WLAN commonly authenticates:

```
user
device
or both
```

before granting appropriate access.

The resulting traffic might be placed into different:

```
VLANs
segments
security policies
```

Wireless controller

Large enterprises may centrally manage many access points.

```
Wireless Controller
  /      |      \
 /      |      \
AP1    AP2    AP3
 )))    )))    )))
```

Modern implementations may use physical, virtual or cloud-managed control architectures.

Roaming

Imagine walking through an office:

AP1 ---- AP2 ---- AP3

Your laptop/phone needs to transition between AP coverage areas while attempting to maintain connectivity.

That is roaming.

Enterprise WLAN therefore has to coordinate:

authentication
mobility
policy
RF/AP management

We do not need deep radio-frequency theory for the networking architecture you are building here.

21. High availability

Networks inevitably fail.

A robust architecture therefore assumes:

links fail
switches fail
routers fail
providers fail
power fails
software fails

High availability tries to prevent a single failure from becoming an outage.

Link redundancy

Instead of:

Router A ----- Router B

use:

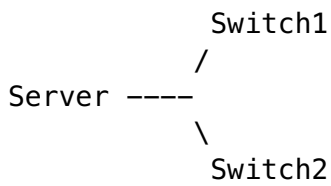
 Link1
Router A ===== Router B
 Link2

Device redundancy

Instead of:

Server → Switch1

possibly:



depending on endpoint/network design.

Active/passive

Router-A = active
Router-B = standby

If A fails:

Router-B takes over

Advantages:

simpler traffic model

Potential disadvantage:

standby resources may be underutilised

Active/active

Both systems actively provide service.

Traffic → Device A
 → Device B

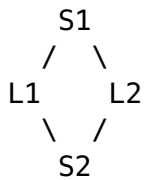
Benefits:

better resource utilisation
higher aggregate capacity

But state consistency and failure behaviour can be more complex.

ECMP as HA

Leaf-spine naturally provides multiple paths.



If S1 fails:

L1 → S2 → L2

Thus ECMP provides both:

capacity
+
resilience

Failover and convergence

Do not confuse them.

Failure detection

"The link died."

Convergence

"Network recalculated/reinstalled usable paths."

Failover

"Traffic is now using the surviving path/system."

BFD may accelerate detection.

OSPF/BGP may reconverge.

ECMP may leave surviving paths already installed.

Resilience vs complexity

There is no free redundancy.

Compare:

1 router
1 ISP
1 link

with:

2 routers
2 firewalls
2 ISPs
4 links
BGP
BFD
ECMP
VRFs
redundant power

The second architecture is much more resilient.

It is also harder to:

configure
test
monitor
troubleshoot
automate
reason about during partial failure

So the engineering principle is:

| Add redundancy where the business requirement justifies its operational complexity.

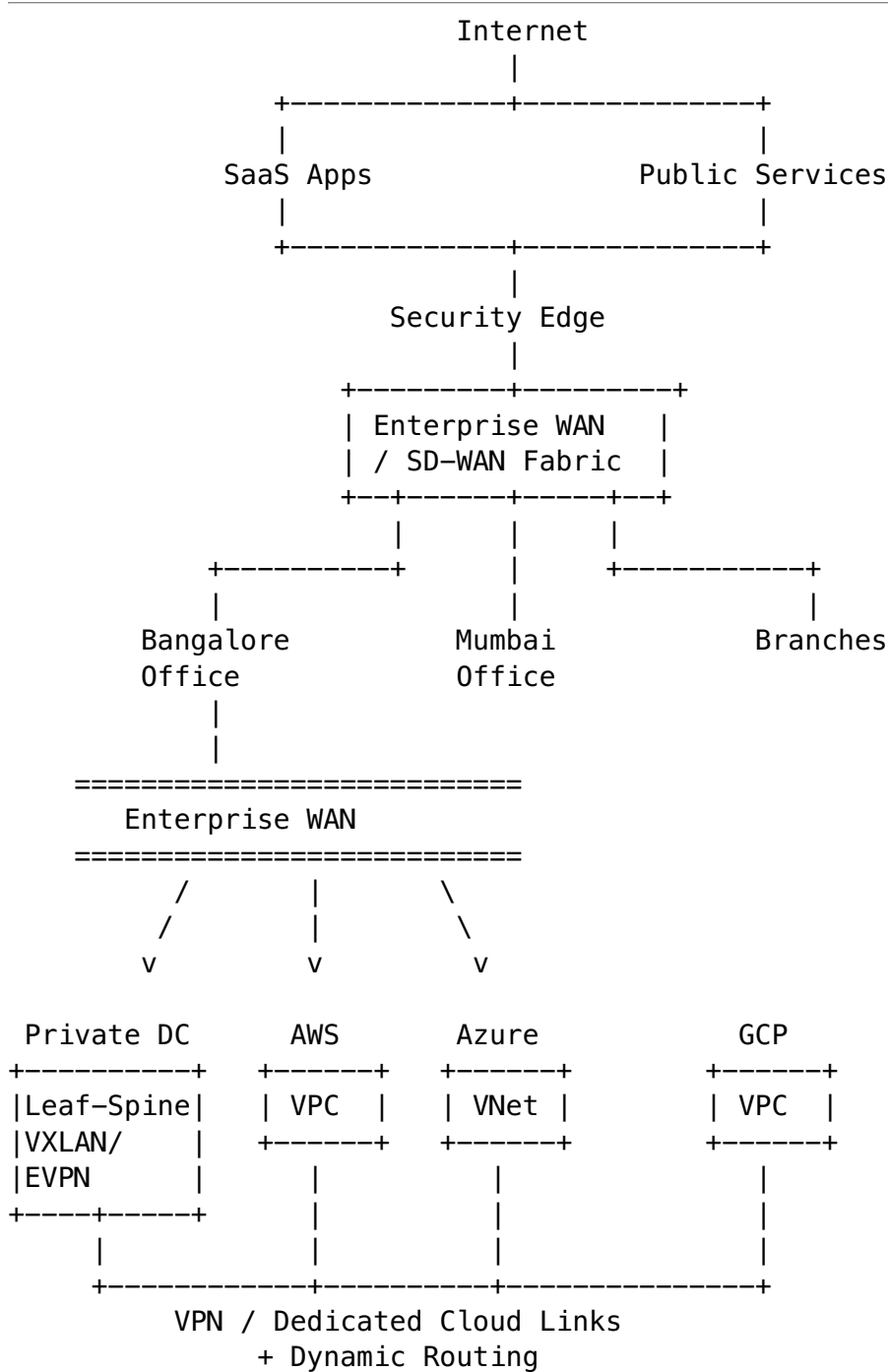
22. Hybrid and multi-cloud networking

Now let's combine everything.

Imagine an enterprise uses:

Bangalore offices
Mumbai offices
Private data centres
AWS
Azure
GCP
SaaS
Public Internet applications

A plausible conceptual architecture is:



Now let's understand every part.

Office connectivity

Branches could have:

Internet
MPLS
5G

and use SD-WAN to build a logical WAN overlay.

Physical transport
↓
SD-WAN overlay
↓
policy-driven connectivity

Private data centre

Inside the data centre:

Leaf-spine
↓
IP underlay
↓
VXLAN overlay
↓
EVPN control plane
↓
VRF-based segmentation

Cloud connectivity

Enterprise-to-cloud connectivity may use:

Site-to-site VPN

Enterprise
||
Internet
||
Cloud

encrypted but Internet dependent.

Dedicated private connectivity

Examples conceptually:

Enterprise
|
Private carrier/dedicated circuit
|
Cloud provider

These can provide more predictable private connectivity than ordinary Internet VPN paths, though designs and guarantees depend on service/provider.

Routing

BGP commonly exchanges routes over these connections.

Enterprise:

10.0.0.0/8



BGP



Cloud

Cloud:

172.16.0.0/12



BGP



Enterprise

Now each side knows relevant reachability.

Segmentation

Not everything should be mutually reachable.

You might have:

Production VRF
Development VRF
Corporate VRF
Security VRF
Management VRF

Cloud environments have their own virtual-networking segmentation mechanisms.

Architecture therefore needs to decide:

which networks can communicate?
through which security boundaries?
using which routes?

Security boundaries

A real hybrid network may include:

```
Internet
|
DDoS/WAF
|
Firewall
|
Application zone
|
Internal firewall/policy
|
Database
```

Or cloud-native equivalents.

Routing answers:

Where can packets go?

Security policy answers:

Should these packets be allowed?

Do not confuse the two.

Centralized vs distributed connectivity

Centralised

Everything passes through a central network hub.

```
AWS ----\
Azure ----+----- Central Hub ---- Enterprise
GCP ----/
```

Advantages:

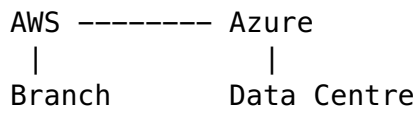
```
central security
central inspection
simpler governance
```

Potential disadvantages:

```
hairpinning
latency
bandwidth bottlenecks
large failure/blast radius
```

Distributed

Some traffic connects directly.



Advantages:

shorter paths
local independence
scaling

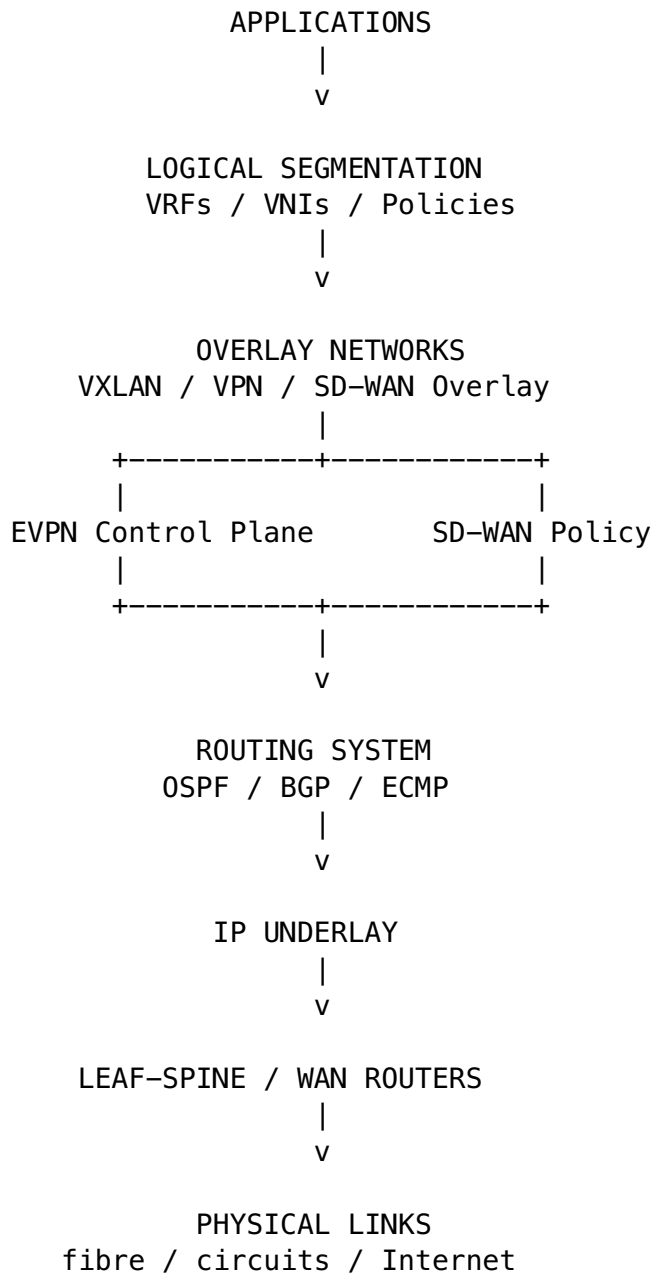
Potential disadvantages:

more routing complexity
more security-policy distribution
harder operational visibility

Modern architectures often combine both models.

Putting all of Day 2 into one mental model

This is the part I would retain above everything else.



And now place every major technology:

Technology	Mental role
Routing	Determine how destinations are reached
OSPF	Internal topology-based routing
BGP	Scalable route distribution + policy
ECMP	Use multiple equivalent paths
BFD	Detect forwarding failures quickly
Leaf-spine	Scalable DC physical/L3 fabric
VRF	Multiple isolated routing tables
MPLS	Provider label-switched transport/VPN foundation
Underlay	Real IP connectivity
Overlay	Logical connectivity above underlay

Technology	Mental role
VLAN	Traditional L2 segmentation
VXLAN	Scalable overlay encapsulation
VNI	Identifies VXLAN logical segment
VTEP	Starts/terminates VXLAN tunnels
EVPN	Distributes endpoint/network reachability
Anycast	Same logical address available from multiple locations
VPN	Secure/private logical tunnel over another network
SD-WAN	Policy-driven WAN overlay using multiple transports
WLAN	Wireless access into enterprise network

The problem → solution evolution

The entire history can be seen as one sequence:

PROBLEM 1

"I have a few networks."

↓

Static routing

↓

PROBLEM 2

"I now have hundreds of changing networks."

↓

Dynamic routing

OSPF / BGP

↓

PROBLEM 3

"I need customer/business isolation."

↓

VLAN + VRF

↓

PROBLEM 4

"I need geographically distributed enterprise connectivity."

↓

WAN / MPLS / VPN

↓

PROBLEM 5

"My data centre has thousands of servers
and huge east-west traffic."

↓

Leaf-Spine + Layer-3 routing + ECMP

↓

PROBLEM 6

"I need far more logical networks than
traditional VLAN architecture scales comfortably for."

↓

VXLAN

↓

PROBLEM 7

"How do VXLAN tunnel endpoints know where endpoints actually live?"

↓

BGP EVPN

↓

PROBLEM 8

"I have MPLS + broadband + Internet + 5G across hundreds of branches."

↓

SD-WAN

↓

PROBLEM 9

"My workloads now exist everywhere: DC + AWS + Azure + GCP + SaaS."

↓

Hybrid / Multi-cloud networking

+

BGP

+

VPN / dedicated connectivity

+

segmentation

+

centralised/distributed architecture

One final end-to-end example

Suppose a developer sitting in the Bangalore office calls an application:

<https://internal-api.company.com>

The application runs in your private data centre.

Behind the API are several microservices distributed across racks.

Follow the traffic.

```
Developer Laptop
  |
  Wi-Fi
  |
Access Point
  |
Enterprise LAN / VRF
  |
SD-WAN Edge
  |
Internet / MPLS transport
  |
SD-WAN Overlay
  |
Data Centre Edge
  |
DC VRF
  |
Leaf Switch
  |
API Server
```

Now the API server needs Service-B on another rack:

```
API
  |
Leaf-1 VTEP
  |
  | VXLAN packet
  |
Spine
  |
Leaf-7 VTEP
  |
Service-B
```

Underneath:

```
BGP/OSPF
  ↓
provide VTEP reachability

ECMP
  ↓
provides several leaf-to-leaf paths
```

Above that:

EVPN

↓

tells Leaf-1 where Service-B's endpoint lives

VXLAN

↓

encapsulates Service-B traffic

VRF/VNI

↓

ensures the workload remains in the correct logical tenant/application network

If a spine link fails:

BFD

↓

may detect failure quickly

Routing

↓

updates/removes path

ECMP

↓

continues across surviving spines

If the enterprise later moves Service-B to AWS, a different portion of the same networking model appears:

Data Centre

|

BGP

|

Dedicated cloud link / VPN

|

AWS

That is why these technologies are connected rather than separate subjects.

The most useful Day-2 architecture sentence is:

The physical network provides resilient IP paths; routing protocols learn those paths; overlays create logical networks above them; VRFs/VNIs provide isolation; EVPN distributes overlay reachability; VXLAN transports overlay traffic; and SD-WAN applies the same overlay-and-policy idea across enterprise WAN links. ([RFC Editor](#))